

Why Deep ResNets Train Successfully: Self-Selection of Effective Depth Enabled by Skip Connections

BMVC 2026 Submission # ??

Abstract

A decade after their introduction, the precise mechanism behind the success of deep Residual Networks (ResNets) remains debated. Why does adding more layers *not* degrade performance, when plain networks of the same depth fail catastrophically? We propose an explanation grounded in direct measurement: **deep ResNets succeed because they autonomously select an effective depth far smaller than their nominal depth, and skip connections are the mechanism that enables this self-selection.** To make this self-selection *directly observable*, we introduce **Learnable Residual Scaling (LRS)**, an analytical instrument that attaches a single learnable scalar α to each residual block: $y = \alpha \cdot F(x) + (1 - \alpha) \cdot x$ with $\alpha = \sigma(\theta) \in (0, 1)$. Because the two coefficients sum to one, the learned α_i directly reports block i 's usage—no post-hoc analysis required. Three lines of evidence support our thesis. (i) *Plain networks cannot self-select and fail*: without skip connections, gradient norms at initialization explode to 10^{13} in early blocks. (ii) *ResNets autonomously compress effective depth*: across CIFAR-10/100 and ImageNet (depths 50–200), the mean $\bar{\alpha}$ decreases monotonically with depth (e.g., $0.34 \rightarrow 0.11$ as depth grows $50 \rightarrow 200$ on CIFAR-100); per-block analysis shows only 5–6 of 66 blocks remain active in a 200-layer network. (iii) *The selection is intrinsic*: identity vs random initialization converge to nearly identical $\bar{\alpha}$, and α -guided pruning validates the measurement—blocks with $\alpha < 0.03$ can be removed with *zero* accuracy loss, and pruning 56% of blocks reduces FLOPs by 54% and latency by 52% at only 4.6% accuracy cost. The depth- α scaling law generalizes to ImageNet, where ResNet-50/101 produce $\bar{\alpha}$ values matching their CIFAR counterparts. This reframes the degradation problem: it is not failure to be solved, but the network correctly refusing to use redundant depth.

1 Introduction

Residual Networks (ResNets) [9] enable the training of networks with hundreds of layers, while plain networks of comparable depth fail catastrophically. The standard explanation—“skip connections stabilize gradient flow”—is technically correct but mechanistically incomplete. Skip connections do stabilize gradients, but this alone does not explain *why deeper ResNets do not degrade*. If skip connections merely allow more computation, deeper networks should be at least as expressive as shallower ones; in practice they are, but only because of *how* that capacity is used. The question we address is the next one: **what does the network do with the depth that skip connections make available to it?**

We propose a concrete answer: *deep ResNets train successfully because they autonomously select an effective depth far smaller than their nominal depth, and skip connections are the architectural mechanism that enables this self-selection.* Adding nominal depth does not commit the network to using all of it—skip connections provide the freedom to refuse. The degradation problem of plain networks is thus not solved by skip connections in the sense of “making all layers trainable”; it is sidestepped by giving the network the option to *choose how deep to be.*

Demonstrating this thesis requires a measurement tool: we need to read off, per block, how much the network actually uses that block. Existing residual scaling methods—ReZero [4], SkipInit [5], Fixup [18], LayerScale [16]—scale only $F(x)$ while leaving the identity at weight 1, which is sufficient for stabilizing training but does not yield a per-block usage scalar. Veit *et al.* [17] inferred shorter-than-nominal effective paths via path-counting and lesion studies, but their evidence is indirect.

Learnable Residual Scaling (LRS). We introduce a minimal modification to the residual block:

$$y = \alpha \cdot F(x) + (1 - \alpha) \cdot x, \quad \alpha = \sigma(\theta) \in (0, 1), \quad (1)$$

where θ is a single learnable scalar per block. Because the two coefficients sum to one, the learned α_i is block i ’s usage: $\alpha_i \rightarrow 0$ means the block is effectively an identity (bypassed), $\alpha_i \rightarrow 1$ means a full transformation. LRS is not a performance method—it is a measurement instrument. Per-block scalars make depth self-selection *directly observable* rather than inferred.

Three lines of evidence. We support the self-selection thesis with empirical evidence organized around three claims.

(a) *Plain networks cannot self-select, hence they fail.* Figure 1 shows per-block gradient L2 norms at initialization for a 200-layer network on CIFAR-100. Without skip connections, the gradient at Block 0 reaches 1.72×10^{13} while Block 65 sits at 5.0—a $3.5 \times 10^{12} \times$ disparity that makes meaningful learning impossible. Skip connections stabilize this disparity to $673 \times$, but our LRS measurements (panels b–c) reveal that the network does not use this stability to leverage all blocks. Instead, gradients in late blocks decay to near zero, reflecting learned $\alpha_i \approx 0$: the network is not attempting to train those blocks at all.

(b) *ResNets autonomously compress effective depth.* Across four depths (50, 101, 152, 200), two CIFAR datasets, and ImageNet at depths 50 and 101 (three seeds each for CIFAR), the mean converged $\bar{\alpha}$ decreases systematically with depth (e.g., $0.34 \rightarrow 0.11$ as depth grows $50 \rightarrow 200$ on CIFAR-100). Per-block analysis shows only 5–6 of 66 blocks maintain $\alpha > 0.3$ in a 200-layer network. The threshold-free metric $D_{\text{eff}} = \sum_i \alpha_i$ confirms that quadrupling blocks ($16 \rightarrow 66$) increases effective depth by only 40% ($5.4 \rightarrow 7.5$). The network is given depth and chooses to ignore most of it.

(c) *The selection is intrinsic, not an artifact.* Three controls argue against optimization-induced bias. First, identity initialization of the residual branch yields nearly identical converged $\bar{\alpha}$ (0.114 vs. 0.118 for random init), showing the result does not depend on the starting point. Second, α -guided block pruning validates the measurement: blocks with $\alpha < 0.03$ can be removed without retraining and incur *zero* accuracy loss; pruning 56% of blocks ($\alpha < 0.08$) followed by 10 epochs of fine-tuning recovers 75.91% from the original 80.54% while reducing FLOPs by 54% and inference latency by 52%. Third, residual-favoring initialization ($\alpha_0 \approx 0.88$) causes catastrophic training failure at depth 200 (CIFAR-100: $37.50 \pm 29.20\%$), confirming that the identity-favored regime is not optional but necessary.

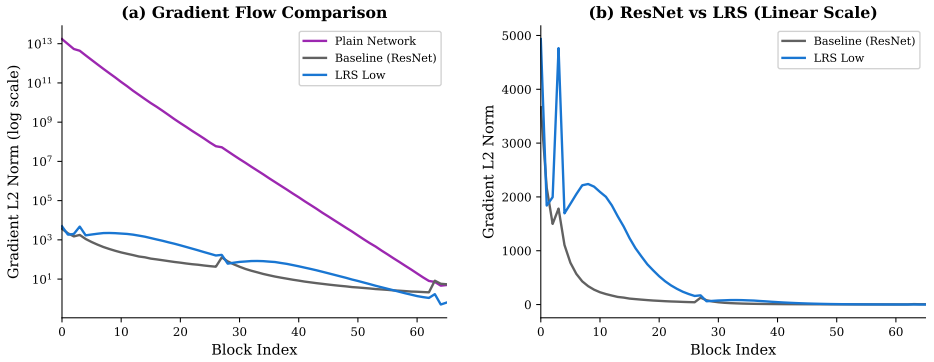


Figure 1: Per-block gradient L2 norms at initialization for ResNet-200 on CIFAR-100. (a) Plain networks without skip connections cannot self-select depth—gradients explode to $> 10^{13}$ in early blocks. Skip connections stabilize the gradient distribution. (b–c) With LRS, late-block gradients decay toward zero, reflecting the network’s learned choice $\alpha_i \approx 0$: those blocks are not used.

Contributions. (1) We propose that ResNets train successfully because of self-selection of effective depth, made possible by skip connections; LRS makes this measurable. (2) We empirically establish a depth– α scaling law that holds across CIFAR-10/100 and ImageNet at multiple depths. (3) We validate that the measurement is semantically meaningful through pruning, identity-init invariance, and initialization sensitivity analyses; α -guided pruning achieves 54% FLOPs reduction with 4.6% accuracy cost. (4) We reframe the degradation problem: it is not failure to be solved, but the network refusing to use depth it does not need.

2 Related Work

2.1 Residual Networks and Skip Connections

ResNet [8] introduced skip connections to alleviate gradient vanishing in deep networks; identity mappings further improve gradient flow [8]. Highway Networks [12] preceded ResNet with gated information flow. De & Smith [10] showed batch normalization biases standard ResNet residuals toward identity—a phenomenon LRS makes explicit but does not create.

2.2 Residual Scaling Methods

Several methods scale the residual branch (Table 1). **ReZero** [11] initializes a scalar multiplier $\alpha = 0$ on the residual branch: $y = x + \alpha F(x)$. **SkipInit** [13] applies the same form with batch normalization retained. **Fixup** [14] initializes the last BN of each block to zero so $F(x) \approx 0$ at start. **LayerScale** [15] introduces per-channel learnable scaling in Vision Transformers. All these methods scale only $F(x)$ while keeping x at weight 1. LRS instead learns the relative weight of both branches via a single scalar, which is the key to per-block usage measurement.

Table 1: Residual scaling methods. Only LRS measures the relative ratio between $F(x)$ and x as a single per-block scalar, enabling direct effective-depth measurement.

Method	Formulation	Per-block measurable	Key difference
ReZero	$y = x + \alpha F(x)$	$ F(x) $ scale	x weight fixed at 1
SkipInit	$y = x + \alpha F(x)$	$ F(x) $ scale	x weight fixed at 1
LayerScale	$y = x + \Lambda F(x)$	per-channel $ F(x) $	x weight fixed at 1
Fixup	$y = x + \beta F(x)$	$ F(x) $ scale	zero-init last layer
LRS (ours)	$y = \alpha F(x) + (1-\alpha)x$	$F(x):x$ ratio	both sides adjusted

2.3 Comparison with Highway Networks

Highway Networks [4] use a similar-looking convex combination $y = T(x)F(x) + (1-T(x))x$ with $T(x) = \sigma(W_g x + b_g)$. The distinction is structural and central to our analysis. Highway’s transform gate $T(x)$ is *input-dependent*: every input produces a different gate value, so per-block usage cannot be summarized by a single number. LRS instead uses an *input-independent scalar* $\alpha = \sigma(\theta)$, deliberately removing input dependency to obtain a per-block measurement. This is a methodological choice for interpretability: a single α per block makes “how much does this block transform its input” a well-defined quantity, enabling the depth- α scaling law, the per-block distribution analysis, and the pruning validation that follow. Highway gates, by contrast, would require aggregating over input distributions to produce a comparable scalar—losing the very interpretability that motivates our design.

2.4 Effective Depth and Network Redundancy

Huang *et al.* [9] proposed stochastic depth, randomly dropping residual blocks during training, showing that not all blocks are needed. The lottery ticket hypothesis [9] demonstrated that large networks contain small subnetworks matching the full network’s performance. The closest prior work is Veit *et al.* [10], which interpreted ResNets as ensembles of paths and showed empirically that effective paths are much shorter than nominal depth. However, prior evidence is *indirect*: path-ensemble analysis counts paths of various lengths, and lesion studies infer importance by deleting blocks and measuring degradation. The question “how much does block i actually contribute” is answered only inferentially. LRS provides a *direct, per-block, quantitative measurement*: the learned α_i values are themselves the blocks’ contributions, observable without any post-hoc analysis. This direct measurability enables (i) a quantitative scaling law $\bar{\alpha}(L)$, (ii) a threshold-free effective-depth metric $D_{\text{eff}} = \sum_i \alpha_i$, and (iii) a principled pruning criterion validated by zero-loss removal at $\alpha < 0.03$.

2.5 Comparison with Trained-Importance Pruning

A related line of work uses learnable scalars within blocks as importance signals for pruning. Network Slimming [11] repurposes Batch Normalization’s γ as a per-channel importance score, applying L_1 regularization to induce sparsity. The mechanism is superficially similar—a learned scalar reveals component usage—but the operand and semantic differ fundamentally on three axes. **Granularity**: Network Slimming measures *per-channel* importance within a fixed depth; LRS measures *per-block* usage and yields a network-level effective depth D_{eff} . **Interpretation**: γ scales a single normalized feature and requires explicit L_1 pressure to become sparse; α is a convex-combination weight whose semantics

(the $F:x$ ratio) are intrinsic—no sparsity prior is applied, and small α emerges from training dynamics alone. **Claim:** Network Slimming is a pruning method; LRS is an analytical instrument whose pruning experiment is a *validation* of the self-selection thesis, not the contribution itself. Channel Pruning [4] and similar trained-importance methods share Network Slimming’s perspective and are likewise complementary to ours.

3 Method

3.1 Learnable Residual Scaling

We redefine the residual block (Eq. 1) where θ is a learnable scalar parameter per block and σ is the sigmoid. The parameterization serves two purposes. First, since $\alpha + (1 - \alpha) = 1$, the *relative ratio* between $F(x)$ and x is directly interpretable: $\alpha = 0.14$ means the block allocates 14% weight to the transformation and 86% to the identity. Second, the bounded range ensures numerical stability. The semantics give a natural definition of effective depth: $\alpha \rightarrow 0$ means the block is effectively bypassed ($y \approx x$); $\alpha \rightarrow 1$ means a full transformation. The effective depth of the network is the number of blocks with α above a meaningful threshold, with $D_{\text{eff}} = \sum_i \alpha_i$ as a threshold-free analogue.

3.2 Gradient Flow Analysis

The backward pass through an LRS block yields:

$$\frac{\partial \mathcal{L}}{\partial x} = \frac{\partial \mathcal{L}}{\partial y} [(1 - \alpha) + \alpha \cdot F'(x)] \quad (2)$$

and the total gradient through L blocks is the product:

$$\frac{\partial \mathcal{L}}{\partial x_0} = \frac{\partial \mathcal{L}}{\partial y_L} \cdot \prod_{i=1}^L [(1 - \alpha_i) + \alpha_i \cdot F'_i(x_i)]. \quad (3)$$

This product structure explains the depth- α relationship. For the product to remain stable across L blocks, each factor must be close to 1. When α_i is small, the factor approaches $(1 - \alpha_i) \approx 1$ regardless of F'_i , ensuring gradient stability. As L grows, smaller α_i values are needed to maintain the product near unity, providing a theoretical basis for why deeper networks should converge to smaller $\tilde{\alpha}$. Conversely, when α_i is large, the factor becomes $\approx F'_i$ and the gradient product degenerates to that of a plain network, prone to vanishing or explosion. This predicts that residual-favoring initialization leads to training collapse at extreme depth—verified in Section 4.5.

3.3 Initialization Variants

We test three initialization strategies (Table 2) to study convergence and sensitivity. These serve as probes: if the optimization landscape has a single basin, all three converge to the same $\tilde{\alpha}$; otherwise, divergence reveals the role of initial conditions.

Table 2: LRS variants by α initialization. The three starting points probe whether α converges to the same optimum.

Variant	θ_0	α_0	Initial bias
LRS Low	-2.0	≈ 0.12	identity-favoring
LRS Mid	0.0	0.50	balanced
LRS High	+2.0	≈ 0.88	residual-favoring

4 Experiments

4.1 Experimental Setup

Datasets. CIFAR-10/100 [10] (10/100 classes, 50K/10K train/test) for systematic analysis; ImageNet [11] (1000 classes, 1.28M/50K) for scale validation.

Architectures. Bottleneck ResNet-50/101/152/200 (16/33/50/66 blocks). CIFAR uses a 3×3 stride-1 stem; ImageNet uses the standard 7×7 stride-2 + max-pool stem [12].

Training. CIFAR: SGD with momentum 0.9, weight decay 10^{-4} ; initial LR 0.1 with cosine annealing [13]; 5-epoch linear warmup; 100 epochs. Batch size is depth-dependent due to GPU memory (128, 64, 32, 32 at depths 50, 101, 152, 200). ImageNet: same optimizer, LR 0.2, batch size 512 (fixed), 60 epochs, 5-epoch warmup. All CIFAR experiments use 3 seeds (42, 123, 456) and report mean $\pm$ std; ImageNet uses a single seed due to compute. The batch-size variation across CIFAR depths is a known confound; however, ImageNet at *fixed* batch size 512 shows the same $\bar{\alpha}$ scaling slope ($d50 \rightarrow d101$: $0.303 \rightarrow 0.166$, 45% drop) as CIFAR-100 with variable batch sizes ($0.342 \rightarrow 0.183$, 46% drop), arguing that batch size is not the driver of the depth- $\bar{\alpha}$ relationship.

Models. Five core variants: Baseline (standard ResNet), LRS Low/Mid/High ($\alpha_0 \in \{0.12, 0.5, 0.88\}$), and ReZero. For method comparison we additionally evaluate SkipInit, Fixup, and LayerScale.

4.2 Depth- α Scaling Law

We first measure the central observation: how does the network’s preferred $\bar{\alpha}$ change with depth? Table 3 reports converged $\bar{\alpha}$ on CIFAR-100 across the three initialization variants (3-seed mean).

Table 3: Mean converged $\bar{\alpha}$ by depth on CIFAR-100 (3-seed average). Low and Mid converge to similar final values; High diverges at extreme depth.

Depth	Blocks	LRS Low	LRS Mid	LRS High
50	16	0.34	0.35	0.36
101	33	0.18	0.20	0.21
152	50	0.13	0.15	0.23
200	66	0.11	0.13	0.28

Several observations emerge. (i) **Monotonic decrease with depth.** The converged $\bar{\alpha}$ decreases from 0.34 (depth 50) to 0.11 (depth 200) for LRS Low—each block’s contribution shrinks as more blocks are added. (ii) **Partial convergence across initializations.** At depth 50, all three variants converge to similar values (0.34–0.36). At depth 200, the High variant

settles at 0.28, significantly above the 0.11–0.13 range of Low/Mid, with severely degraded performance (Section 4.5). (iii) **Dataset independence.** CIFAR-10 yields nearly identical $\bar{\alpha}$ at each depth, suggesting an architectural rather than data-driven property.

Verification at scale. To confirm the law generalizes beyond CIFAR, we measure $\bar{\alpha}$ on ImageNet at two depths (Figure 2). ResNet-50 yields $\bar{\alpha} = 0.303$, ResNet-101 yields $\bar{\alpha} = 0.166$. Both values closely track their CIFAR-100 counterparts (0.342, 0.183) and follow the same scaling trend. ImageNet has $\sim 25\times$ more training samples, $10\times$ more classes, and $7\times$ larger input resolution than CIFAR; the persistence of the α pattern across this gap rules out simple dataset-size explanations.

Although ImageNet uses a single seed due to compute, the CIFAR scaling law has tight cross-seed variance: $\bar{\alpha}$ std at depths (50, 101, 152, 200) is (0.0007, 0.0005, 0.0012, 0.0002)—all below 1% of the mean. The ImageNet–CIFAR-100 gaps at matched depths are $0.342 - 0.303 = 0.039$ at d_{50} and $0.183 - 0.166 = 0.017$ at d_{101} , both $20\text{--}80\times$ larger than the CIFAR seed-noise scale, indicating a small but systematic dataset shift while preserving monotonicity. The ratio of ImageNet $\bar{\alpha}$ at $d_{50} \rightarrow d_{101}$ is $0.303/0.166 = 1.83$, essentially matching the CIFAR-100 ratio $0.342/0.183 = 1.87$; the scaling law’s slope is preserved across the dataset gap. A statistical accident producing the same decreasing trend across two depths at this gap-to-noise ratio is implausible.

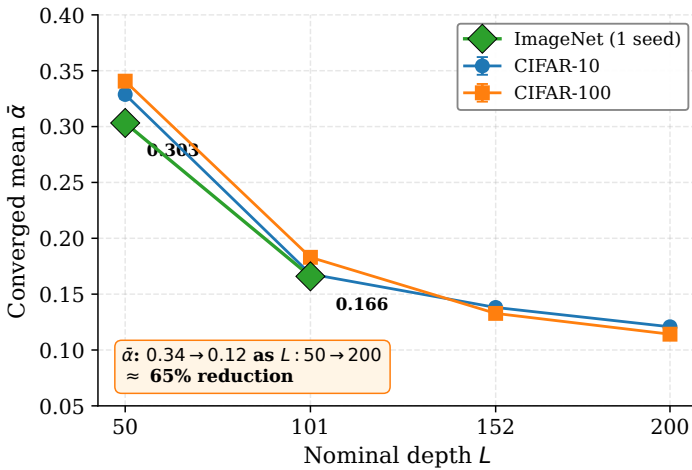


Figure 2: The depth– $\bar{\alpha}$ scaling law. Mean converged $\bar{\alpha}$ decreases monotonically with nominal depth L across CIFAR-10, CIFAR-100, and ImageNet. As depth grows $50 \rightarrow 200$, ResNets autonomously suppress $\sim 65\%$ of their residual contribution. Error bars: ± 1 std over 3 seeds (CIFAR); ImageNet uses a single seed.

4.3 Per-Block α Distribution

The mean hides structure. Figure 3 shows per-block α for a 200-layer network (66 blocks, LRS Low, CIFAR-100). Two patterns emerge: **downsampling blocks have high α** (0.42–0.96), and **normal blocks have low α** (0.06–0.15) with progressive decrease toward stage end. The network treats each stage as “one essential transformation (downsampling) followed by progressively negligible refinements.”

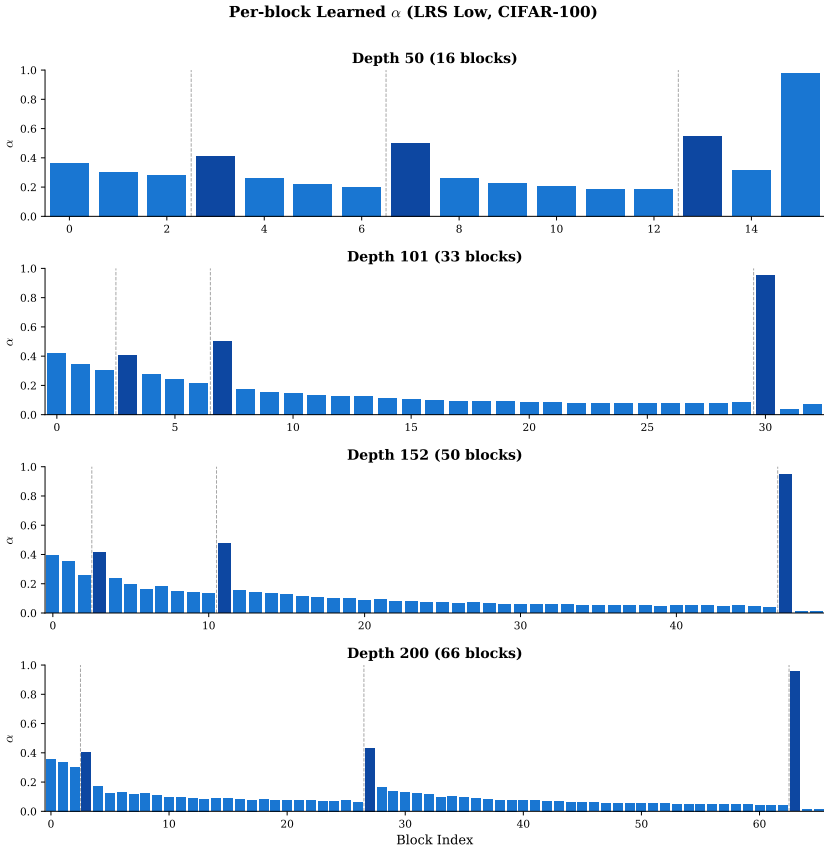


Figure 3: Per-block α for ResNet-200 on CIFAR-100. Stage boundaries (downsampling blocks) maintain high α ; within-stage blocks converge to near-zero values.

Table 4: Threshold-free effective depth $D_{\text{eff}} = \sum_i \alpha_i$ for LRS Low on CIFAR-100. Although nominal depth scales $4\times$, D_{eff} scales only $1.4\times$, confirming that most added blocks are not used.

Depth	Blocks (N)	$D_{\text{eff}} = \sum \alpha_i$	D_{eff}/N	Active ($\alpha > 0.3$)
50	16	5.4	34.0%	6–7
101	33	6.0	18.3%	6
152	50	6.7	13.3%	5–6
200	66	7.5	11.4%	5–6

4.4 Effective Depth Quantification

Using $\alpha > 0.3$ as “active block” threshold, only 5–6 of 66 blocks in a 200-layer network qualify—effective depth $\sim 8\%$ of nominal. To avoid threshold dependence, we additionally report the threshold-free metric $D_{\text{eff}} = \sum_i \alpha_i$ (Table 4): quadrupling blocks ($16 \rightarrow 66$) increases D_{eff} by only 40% ($5.4 \rightarrow 7.5$). Per-block decomposition shows that within-stage $\bar{\alpha}$ stays in $[0.06, 0.15]$ (near-identity) while downsampling-block $\bar{\alpha}$ remains in $[0.42, 0.96]$ (active transformation).

4.5 Initialization Sensitivity

The three initializations reveal dramatic differences in training stability with depth (Tables 5, 6). LRS Low matches or beats baseline at all depths with low variance. LRS Mid degrades at depth 200. LRS High shows progressive degradation, culminating in catastrophic failure (CIFAR-100: $37.50 \pm 29.20\%$). The mechanism is gradient-preservation (Eq. 3): with $\alpha_0 = 0.88$, the identity baseline is $0.12^{66} \approx 0$ through 66 blocks, making learning impossible without rapid α adjustment—which itself requires gradients.

Table 5: CIFAR-10 test accuracy (% , 3-seed mean $\pm$ std). LRS High collapses at depth 200.

Depth	Baseline	LRS Low	LRS Mid	LRS High	ReZero
50	94.53 $\pm$ 0.14	94.83 $\pm$ 0.13	94.77 $\pm$ 0.03	94.27 $\pm$ 0.17	94.60 $\pm$ 0.28
101	95.37 $\pm$ 0.11	95.44 $\pm$ 0.12	95.15 $\pm$ 0.11	94.28 $\pm$ 0.29	95.38 $\pm$ 0.21
152	95.95 $\pm$ 0.06	95.73 $\pm$ 0.10	95.60 $\pm$ 0.15	93.78 $\pm$ 0.87	95.99 $\pm$ 0.06
200	95.98 $\pm$ 0.15	96.10 $\pm$ 0.12	94.36 $\pm$ 0.92	67.25 $\pm$ 43.65	96.02 $\pm$ 0.16

Table 6: CIFAR-100 test accuracy (% , 3-seed mean $\pm$ std). LRS High fails catastrophically.

Depth	Baseline	LRS Low	LRS Mid	LRS High	ReZero
50	77.18 $\pm$ 0.36	77.02 $\pm$ 0.20	76.72 $\pm$ 0.40	75.22 $\pm$ 0.45	76.12 $\pm$ 0.09
101	79.29 $\pm$ 0.23	78.85 $\pm$ 0.14	78.34 $\pm$ 0.19	75.16 $\pm$ 1.69	78.04 $\pm$ 0.48
152	80.42 $\pm$ 0.22	80.28 $\pm$ 0.38	78.57 $\pm$ 0.60	71.66 $\pm$ 3.40	79.72 $\pm$ 0.10
200	80.72 $\pm$ 0.17	80.65 $\pm$ 0.01	76.07 $\pm$ 1.59	37.50 $\pm$ 29.20	80.14 $\pm$ 0.17

4.6 Intrinsic vs. Artifact: Identity Initialization Test

A natural concern: is the low effective depth an artifact of randomly initialized residual branches? We test this by initializing residual branches as identity transformations (HybridA: identity init for layers 3 & 4; Full Identity: all layers). Results on CIFAR-100 (Table 7): full identity init catastrophically fails (12.98% at depth 200) due to symmetry that the network cannot break. Partial identity + LRS achieves the best accuracy (81.14%). *Crucially*, the learned $\tilde{\alpha}$ is nearly identical between random and identity init (0.114 vs. 0.118, with 5 vs. 6 active blocks out of 66). The network converges to the same effective depth regardless of where the residual branch starts—ruling out initialization as the source of low effective depth.

Table 7: Identity init analysis (CIFAR-100, 3-seed mean $\pm$ std). Effective depth is invariant to residual branch initialization.

Model	d152 (%)	d200 (%)	$\tilde{\alpha}$ (d200)	Active / 66
Baseline (He init)	80.42 $\pm$ 0.18	80.72 $\pm$ 0.13	—	—
LRS Low + He init	80.28 $\pm$ 0.31	80.65 $\pm$ 0.01	0.114	5
HybridA (Identity L3,4)	80.02 $\pm$ 0.26	80.40 $\pm$ 0.17	—	—
LRS Low + HybridA	80.72 $\pm$ 0.29	81.14 $\pm$ 0.18	0.118	6
ResNet (Full Identity)	13.41 $\pm$ 0.41	12.98 $\pm$ 0.16	—	—

4.7 Fixed vs. Learned α

To validate that *learning* α is essential, we compare with fixed- α variants at depth 152 (Table 8). Two findings emerge. (i) Fixed $\alpha = 0.5$ yields only 44.19% because the formulation $0.5F(x) + 0.5x$ halves both branches uniformly, unlike standard ResNet’s $F(x) + x$. We include this point for completeness of the sweep; the resulting drop confounds the role of magnitude with the role of allocation and should not be read as the load-bearing comparison. (ii) The relevant comparison is fixed $\alpha = 0.1$, close to the learned mean, which still underperforms learned LRS by 0.47%. While modest, this gap is consistent with learned LRS exploiting per-block heterogeneity: downsampling blocks maintain $\alpha \in [0.42, 0.96]$ while within-stage blocks settle near $\alpha \in [0.06, 0.15]$ (Section 4.3). A uniform constant cannot capture this structure.

Table 8: Fixed vs. learned α at depth 152, CIFAR-100. Learned per-block structure is essential.

Configuration	Accuracy (%)	Notes
Learned (LRS Low)	80.28	per-block α , mean ≈ 0.13
Fixed $\alpha = 0.1$	79.81	close to learned mean
Fixed $\alpha = 0.3$	67.94	moderate transformation
Fixed $\alpha = 0.5$	44.19	halves both branches
Fixed $\alpha = 0.7$	20.62	residual-favoring
Baseline ($F(x)+x$)	80.42	no LRS

4.8 α -Guided Pruning and Efficiency

If α_i truly reports block usage, then blocks with very low α should be removable without harm—a direct empirical test of our thesis. We prune blocks with $\alpha < \tau$ for various thresholds, replacing them with identity, and measure accuracy, FLOPs, and inference latency (NVIDIA A100 GPU, batch size 1).

Table 9 reports the results on ResNet-200/CIFAR-100. Three findings:

Table 9: α -guided pruning on ResNet-200/CIFAR-100. Accuracy retained through 10 epochs of fine-tuning; FLOPs and latency measured on actual short-circuited forward pass.

Threshold	Removed	Acc no-FT	Acc +FT	■Acc	FLOPs	Latency
None (orig.)	0 / 66	80.54%	—	—	100.0%	100.0%
$\alpha < 0.03$	2 / 66	80.54%	80.55%	+0.01	97.1%	97.7%
$\alpha < 0.05$	10 / 66	73.88%	79.65%	−0.89	85.4%	86.1%
$\alpha < 0.08$	37 / 66	3.97%	75.91%	−4.63	45.8%	48.5%
$\alpha < 0.10$	48 / 66	3.13%	74.74%	−5.80	29.7%	32.3%
$\alpha < 0.15$	58 / 66	3.63%	72.63%	−7.91	15.0%	18.0%

(a) **Zero-loss pruning at $\alpha < 0.03$.** Two blocks (12 conv/BN layers) can be removed *without* retraining and *without* accuracy loss. This is a direct semantic validation: blocks the network labels as $\alpha \approx 0$ truly contribute nothing.

(b) **Practical efficiency gains.** At $\alpha < 0.08$, 56% of blocks can be removed with 54% FLOPs and 52% latency reduction at a cost of 4.6% accuracy after fine-tuning—a strong

efficiency-accuracy trade-off. At $\alpha < 0.10$ (73% pruned), the network retains 74.74% accuracy with only 30% of the original FLOPs.

(c) Self-selection vs. minimum sufficient depth. Without fine-tuning, removing $\alpha < 0.08$ blocks collapses accuracy to 3.97%, but 10 epochs of fine-tuning recover it to 75.91%. At first glance this might appear to contradict the “5–6 essential blocks” claim. It does not. The α_i values measure each block’s *learned allocation under the original training*—they identify which blocks the network *chose* to use, not which blocks are *uniquely capable* of carrying that role. The 5–6 active blocks are essential *conditional on the trained configuration*: removing them without fine-tuning collapses accuracy to 3.97%. The recovery via fine-tuning is a separate property of the network’s *capacity reserve*: 60 dormant blocks, each individually near-identity, can collectively re-learn the missing computation when forced to. Self-selection is thus a statement about *learned* effective depth, not about *minimum sufficient* depth—and the gap between them is precisely the “reserve” that skip connections preserve as latent capacity.

Figure 4 visualizes the accuracy-efficiency trade-off.

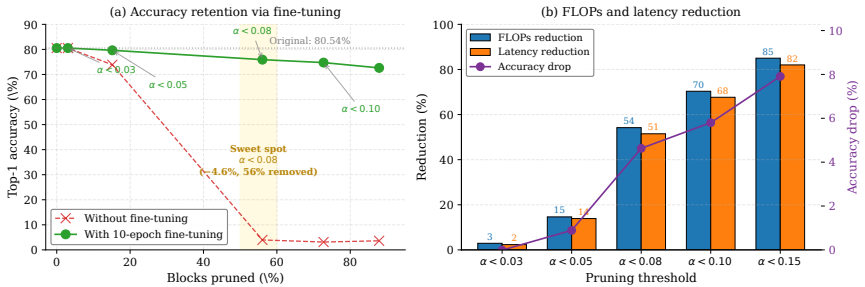


Figure 4: α -guided pruning on ResNet-200/CIFAR-100. (a) Fine-tuning recovers accuracy across all thresholds; the sweet spot is $\alpha < 0.08$ at 4.6% drop. (b) Pruning $\alpha < 0.08$ removes 56% of FLOPs and 51% of inference latency, demonstrating that self-selected low- α blocks are functionally near-identity.

4.9 Comparison with Existing Methods

We compare LRS with residual scaling baselines at depth 152 on CIFAR-100 (Table 10, 3-seed mean). All methods fall within a 0.84% band. Performance similarity is expected—this is not a method paper. LRS distinguishes itself in providing per-block interpretability (Table 1) at no accuracy cost.

Table 10: Residual scaling methods at depth 152, CIFAR-100 (3-seed mean). Methods perform similarly; LRS uniquely provides per-block measurement.

Method	Accuracy (%)
LayerScale	80.56
Baseline	80.42
LRS Low	80.28
Fixup	79.79
SkipInit / ReZero	79.72

5 Discussion

5.1 ResNets Self-Select Their Depth

The central finding is that deep ResNets do not use their full depth. A 200-layer network concentrates transformations in 5–6 blocks—primarily at stage boundaries—and reduces the remaining 60 to near-identity. This is not an artifact of LRS measurement: identity initialization yields the same result, and pruning validation confirms that $\alpha < 0.03$ blocks truly contribute nothing.

The conventional view is that skip connections solve the degradation problem by enabling gradient flow through deep networks. Our gradient analysis (Figure 1) confirms gradient stabilization. However, LRS reveals that the network uses this stability *not* to leverage deep computation but to *bypass* most blocks. The reframing: degradation is not a problem to be solved but a natural consequence of the network selecting its optimal depth. Skip connections provide the *mechanism* for this self-selection by enabling blocks to act as near-identity without disrupting gradient flow.

This resonates with the unrolled-iterative-estimation view [9] and the path ensemble interpretation [10]. Our contribution is to make self-selection directly observable through α rather than inferring it from paths or lesions.

Relation to Stochastic Depth. Stochastic Depth [9] demonstrates that random per-iteration block dropping during training preserves accuracy. Our finding is the converse and complementary: given a continuous knob, the network itself converges to a *deterministic* near-identity for the same blocks that Stochastic Depth could randomly drop without harm. The two measure the same phenomenon—within-stage redundancy—from opposite directions (input-side noise vs. learned weight), and their agreement is mutual evidence. The distributed redundancy we observe under fine-tuning recovery (Section 4.8) is precisely what makes random dropping viable: blocks are not individually irreplaceable, but the network collectively allocates active roles to a small subset under standard training.

5.2 The Role of Downsampling Blocks

Per-block analysis shows that downsampling blocks maintain high α (0.42–0.96), while within-stage blocks converge to $\alpha < 0.15$. This grounds a design principle implicit in many efficient architectures: *the number of stages (and thus downsampling operations) matters more than the total number of blocks per stage*. Three lines of prior work converge with this. MobileNet [8] and EfficientNet [11] achieve strong accuracy with shallow per-stage block counts. Stochastic Depth [9] found dropping within-stage blocks hurts less than dropping stage-transition blocks. NAS literature [12] consistently allocates more channels at stage transitions than additional within-stage layers. LRS explains *why*: the within-stage blocks these methods de-emphasize are precisely the blocks our α measurements flag as near-identity.

5.3 Why Standard ResNet Still Works Well

If the optimal α is much less than 0.5, why does standard ResNet (implicit $\alpha = 0.5$) perform well? Standard ResNet uses $y = F(x) + x$, which does not constrain $\|F(x)\|/\|x\|$. Through batch normalization and weight magnitude adjustment, the network can effectively achieve a

small ratio without an explicit α —a phenomenon De & Smith [2] established independently. LRS makes this implicit ratio observable and quantifies it.

5.4 Practical Implications

Our findings translate into three actionable guidelines. **(1) α -guided pruning** provides an automatic block-importance criterion produced as a training byproduct—one-shot workflow (train once with LRS, threshold α , fine-tune ~ 10 epochs) yields 54% FLOPs / 52% latency reduction at 4.6% accuracy cost. **(2) Identity-favoring initialization** ($\alpha_0 \leq 0.12$, $\theta_0 \leq -2$) is necessary at extreme depth—residual-favoring initialization actively destabilizes training. **(3) Architecture design:** the heterogeneous α distribution justifies allocating capacity at stage boundaries rather than within stages, and NAS procedures could use α -statistics as a regularizer against over-deep within-stage configurations.

5.5 Limitations

ImageNet scale. We verify the scaling law at ImageNet depths 50 and 101 with single seeds; deeper models and additional seeds would strengthen the claim. The CIFAR seed-noise floor ($< 1\%$ of the mean) and the $20\text{--}80\times$ larger ImageNet-CIFAR gaps argue against seed accidents, but ImageNet’s internal noise is not directly characterized. **Measurement bias.** LRS adds one parameter per block; identity-init experiments (Section 4.6) show convergent $\bar{\alpha}$ regardless of starting point, arguing intrinsic convergence. **Architecture scope.** The analysis is restricted to bottleneck ResNets; basic blocks and Vision Transformers are not covered. **Threshold dependency.** The “active block” threshold $\alpha > 0.3$ is arbitrary, addressed via the threshold-free $D_{\text{eff}} = \sum_i \alpha_i$.

6 Conclusion

We proposed and empirically supported a concrete answer to why deep ResNets train successfully: they autonomously select an effective depth far smaller than their nominal depth, and skip connections provide the architectural freedom that makes this self-selection possible. Across 120 CIFAR runs and ImageNet at depths 50/101, deep ResNets compress effective depth to roughly 8% of nominal; the depth- $\bar{\alpha}$ scaling law is consistent across datasets. Skip connections are the enabling mechanism: without them gradients explode and self-selection is impossible. The selection is intrinsic, validated by random/identity-init invariance ($\bar{\alpha}$ within 4%) and by α -guided pruning— $\alpha < 0.03$ blocks remove with zero accuracy loss, and 56% block pruning yields 54% FLOPs / 52% latency reduction at 4.6% accuracy cost. The conventional view casts deep networks as “working” only when each added layer contributes positively. Our results suggest a different reading: deep networks work when they are allowed not to use most layers. Capacity not used is not wasted—it is reserve, and skip connections make reserve a viable design choice.

References

- [1] Thomas Bachlechner, Bodhisattwa Prasad Majumder, Henry Mao, Garrison Cottrell, and Julian McAuley. Rezero is all you need: Fast convergence at large depth. In *UAI*, pages 1352–1361, 2021.

- [2] Soham De and Samuel L Smith. Batch normalization biases residual blocks towards the identity function in deep networks. In *NeurIPS*, 2020. 598 599
- [3] Jonathan Frankle and Michael Carbin. The lottery ticket hypothesis: Finding sparse, trainable neural networks. In *ICLR*, 2019. 600 601 602
- [4] Klaus Greff, Rupesh K Srivastava, and Jürgen Schmidhuber. Highway and residual networks learn unrolled iterative estimation. In *ICLR*, 2017. 603 604 605
- [5] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *CVPR*, pages 770–778, 2016. 606 607
- [6] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Identity mappings in deep residual networks. In *ECCV*, pages 630–645, 2016. 608 609 610
- [7] Yihui He, Xiangyu Zhang, and Jian Sun. Channel pruning for accelerating very deep neural networks. In *ICCV*, pages 1389–1397, 2017. 611 612
- [8] Andrew G Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, and Hartwig Adam. Mobilenets: Efficient convolutional neural networks for mobile vision applications. *arXiv preprint arXiv:1704.04861*, 2017. 613 614 615 616 617
- [9] Gao Huang, Yu Sun, Zhuang Liu, Daniel Sedra, and Kilian Q Weinberger. Deep networks with stochastic depth. In *ECCV*, pages 646–661, 2016. 618 619
- [10] Alex Krizhevsky. Learning multiple layers of features from tiny images. *Technical Report, University of Toronto*, 2009. 620 621 622
- [11] Zhuang Liu, Jianguo Li, Zhiqiang Shen, Gao Huang, Shoumeng Yan, and Changshui Zhang. Learning efficient convolutional networks through network slimming. In *ICCV*, pages 2736–2744, 2017. 623 624 625
- [12] Ilya Loshchilov and Frank Hutter. Sgdr: Stochastic gradient descent with warm restarts. *ICLR*, 2017. 626 627 628
- [13] Olga Russakovsky, Jia Deng, Hao Su, Jonathan Krause, Sanjeev Satheesh, Sean Ma, Zhiheng Huang, Andrej Karpathy, Aditya Khosla, Michael Bernstein, et al. Imagenet large scale visual recognition challenge. volume 115, pages 211–252, 2015. 629 630 631
- [14] Rupesh Kumar Srivastava, Klaus Greff, and Jürgen Schmidhuber. Highway networks. *arXiv preprint arXiv:1505.00387*, 2015. 632 633 634
- [15] Mingxing Tan and Quoc Le. Efficientnet: Rethinking model scaling for convolutional neural networks. In *ICML*, pages 6105–6114, 2019. 635 636
- [16] Hugo Touvron, Matthieu Cord, Alexandre Sablayrolles, Gabriel Synnaeve, and Hervé Jégou. Going deeper with image transformers. In *ICCV*, pages 32–42, 2021. 637 638 639
- [17] Andreas Veit, Michael J Wilber, and Serge Belongie. Residual networks behave like ensembles of relatively shallow networks. In *NeurIPS*, 2016. 640 641
- [18] Hongyi Zhang, Yann N Dauphin, and Tengyu Ma. Fixup initialization: Residual learning without normalization. In *ICLR*, 2019. 642 643